

DSA Assignment D: 20260527 模拟考

2500010774 兰玉琪 数学科学学院

1. 题目

M27351:01 最小生成树

补图的连通分量, <http://cs101.openjudge.cn/practice/27351>

思路:

代码:

```
from collections import deque
n, m = map(int, input().split())
bad = [set() for _ in range(n + 1)]
for _ in range(m):
    u, v = map(int, input().split())
    bad[u].add(v)
    bad[v].add(u)
unvisited = set(range(1, n + 1))
cc = 0
while unvisited:
    start = unvisited.pop()
    cc += 1
    q = deque([start])
    while q:
        u = q.popleft()
        nxt = []
        for v in unvisited:
            if v not in bad[u]:
                nxt.append(v)
        for v in nxt:
            unvisited.remove(v)
            q.append(v)
print(cc - 1)
```

M30910:邮递员送快递

正向/ 反向图 Dijkstra, <http://cs101.openjudge.cn/practice/30910>

思路:

代码:

```
import heapq
n,m = map(int,input().split())
graphzheng = [[] for _ in range(n)]
graphfan = [[] for _ in range(n)]
for _ in range(m):
    a,b,c = map(int,input().split())
    a -= 1
    b -= 1
    graphzheng[a].append((b,c))
    graphfan[b].append((a,c))
def dijkstra(graph):
    total = 0
    dist = [float("inf")]*n
    dist[0] = 0
    pq = [(0,0)]
    while pq:
        cur,node = heapq.heappop(pq)
        if cur > dist[node]:
            continue
        for nei,w in graph[node]:
            new = cur + w
            if new < dist[nei]:
                dist[nei] = new
                heapq.heappush(pq,(new,nei))
    return sum(dist)
print(dijkstra(graphzheng)+dijkstra(graphfan))
```

M30912:累加树

构建 BST + 右-根- 左累加 + BFS 输出, <http://cs101.openjudge.cn/practice/30912>

思路:

代码:

```

from collections import deque
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.lei = 0
n = int(input())
mylist = list(map(int, input().split()))
def build(i, j):
    if i == j:
        return TreeNode(mylist[i])
    node = TreeNode(mylist[i])
    if mylist[i+1] <= mylist[i]:
        t = i+1
        for k in range(i+1, j+1):
            t = k
            if mylist[k] > mylist[i]:
                break
        if mylist[t] > mylist[i]:
            node.left = build(i+1, t-1)
            node.right = build(t, j)
        else:
            node.left = build(i+1, j)
    else:
        node.right = build(i+1, j)
    return node
root = build(0, n-1)
cur = 0
stack = []
def zhongxu(node):
    global cur
    if node:
        zhongxu(node.right)
        while stack:
            if stack[-1].val != node.val:
                stack.pop()
            else:
                break
        stack.append(node)
        cur += node.val
    for t in stack:
        t.lei = cur

```

```
        zhongxu(node.left)
zhongxu(root)
def cengxu(root):
    queue = deque()
    queue.append(root)
    res = []
    while queue:
        node = queue.popleft()
        res.append(node.lei)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return res
print(*cengxu(root))
```

M30899:火星大工程

关键路径, <http://cs101.openjudge.cn/practice/30899>

思路:

代码:

```

from collections import defaultdict, deque
n, m = map(int, input().split())
edges = set()
for _ in range(m):
    a, b, c = map(int, input().split())
    a -= 1
    b -= 1
    edges.add((a, b, c))
def critical_path(n, edges):
    graph = defaultdict(list)
    in_degree = [0]*n
    for u, v, w in edges:
        graph[u].append((v, w))
        in_degree[v] += 1
    queue = deque()
    for i in range(n):
        if in_degree[i] == 0:
            queue.append(i)
    topo_order = []
    ve = [0]*n
    while queue:
        u = queue.popleft()
        topo_order.append(u)
        for v, w in graph[u]:
            ve[v] = max(ve[v], ve[u]+w)
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)
    vl = [max(ve)]*n
    for u in reversed(topo_order):
        for v, w in graph[u]:
            vl[u] = min(vl[u], vl[v]-w)
    print(max(ve))
    critical_edges = []
    for u, v, w in edges:
        e = ve[u]
        l = vl[v]-w
        if e == l:
            critical_edges.append((u+1, v+1))
    critical_edges.sort()
    for u, v in critical_edges:

```

```
print(u,v)
critical_path(n,edges)
```

T30868:upstairs

同余最短路, <http://cs101.openjudge.cn/practice/30868>

思路：请codex老师帮忙

代码

```

import sys
import heapq
def main():
    data = sys.stdin.read().strip().split()
    it = iter(data)
    a = int(next(it))
    b = int(next(it))
    c = int(next(it))
    q = int(next(it))
    if q == 0:
        return
    queries = [int(next(it)) for _ in range(q)]
    coins = []
    for v in (a, b, c):
        if v > 0:
            coins.append(v)
    coins = sorted(set(coins))
    if not coins:
        out = []
        for h in queries:
            out.append("Yes" if h == 0 else "No")
        print("\n".join(out_lines))
        return
    def gcd(x, y):
        while y:
            x, y = y, x % y
        return x
    g = coins[0]
    for v in coins[1:]:
        g = gcd(g, v)
    if len(coins) == 1:
        step = coins[0]
        out = []
        for h in queries:
            if h == 0 or (step > 0 and h % step == 0):
                out.append("Yes")
            else:
                out.append("No")
        print("\n".join(out_lines))
        return
    # Scale by gcd so the gcd becomes 1 for the residue calculations.
    scaled_coins = [v // g for v in coins]
    smallest = scaled_coins[0]

```

```

# Dijkstra on residues modulo smallest coin.
INF = 10**30
dist = [INF] * smallest
dist[0] = 0
heap = [(0, 0)]
edges = [v for v in scaled_coins if v != smallest]
while heap:
    d, r = heapq.heappop(heap)
    if d != dist[r]:
        continue
    for w in edges:
        nr = (r + w) % smallest
        nd = d + w
        if nd < dist[nr]:
            dist[nr] = nd
            heapq.heappush(heap, (nd, nr))
out_lines = []
for h in queries:
    if h == 0:
        out_lines.append("Yes")
        continue
    if h % g != 0:
        out_lines.append("No")
        continue
    h_scaled = h // g
    r = h_scaled % smallest
    if h_scaled >= dist[r]:
        out_lines.append("Yes")
    else:
        out_lines.append("No")
print("\n".join(out_lines))
if __name__ == '__main__':
    main()

```

T30921:猫猫搭积木

并查集, <http://cs101.openjudge.cn/practice/30921>

思路：请codex老师帮忙

代码


```

import sys
def main():
    data = sys.stdin.read().split()
    it = iter(data)
    n = int(next(it))
    q = int(next(it))
    s = int(next(it))
    parent = list(range(n + 1))
    size = [1] * (n + 1)
    members = [[] for _ in range(n + 1)]
    for i in range(1, n + 1):
        members[i].append(i)

    pile_count = n
    out = []

    def find(i):
        if (parent[i] == i):
            return i
        else:
            result = find(parent[i])
            parent[i] = result
            return result

    for _ in range(q):
        x = int(next(it))
        y = int(next(it))
        rx = find(x)
        ry = find(y)

        if rx != ry:
            if len(members[rx]) < len(members[ry]):
                rx, ry = ry, rx
            parent[ry] = rx
            members[rx].extend(members[ry])
            members[ry].clear()
            size[rx] += size[ry]
            pile_count -= 1
            if size[rx] >= s:
                collapsed = members[rx]
                pile_count += size[rx] - 1
                for block in collapsed:
                    parent[block] = block

```

```
        size[block] = 1
        members[block] = [block]
        collapsed.clear()

    out.append(str(pile_count))

sys.stdout.write("\n".join(out))
```

```
if __name__ == "__main__":
    main()
```

2. 学习总结和个人收获

由于是自愿提交就懒得截accepted的图了，我这点诚信还是有的

本人水平太差，只能ac2，考试完提交了一下累加树，一下就过了，笑死了，算ac3安慰一下自己吧
这次考试坑点非常多

首先是那个dijkstra的那个题，绝对不能用graph[a][b]，必须老老实实给我append，因为边数少
关键路径的那个题就是要注意可能有不连通的，所以应该找max而不是拓扑序最后的那个
累加树没啥好说的

其他就要么不会要么没看